

ntiators for the value play.

Technical Resources for Solution Architects

Sometimes customers and prospects have unique requirements, often around using an preferred scanning tool to integrating with some other part of their tool chain. If you or your customer has a third party they'd like to see integrated into GitLab, send them to the partner integration page for instructions. While GitLab can be a single platform to meet all of their needs, often they need an on-ramp to help them transition or proof of the integration before purchasing GitLab. The resources below may help.

NOTE: Please do not use these to put GitLab in the position where users expect us to support a 3rd-party product integration that we do not officially recognize.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/devops-platform/message-house/index.md

Overview

In the spirit of Command of the Message, this page will help GitLab team members understand and define the GitLab DevOps Platform solutions to customers in an authentic, consistent manner than differentiates us from our competitors. It includes:

- Historical context of the market evolution
- General messaging guidelines
- Third-party validation and perspective
- Segment-specific messaging suggestions

NOTE: If you are searching for approved copy to describe The DevOps Platform, please skip ahead to Messaging Guidelines.

The Four Phases of DevOps

As DevOps has evolved, so has its complexity. This complexity is driven by two factors:

1. Organizations are moving from monolithic architectures to microservices architectures. By using microservices, teams can work independently and move faster.
1. As DevOps matures, organizations need more and more DevOps tools per project.

The result of more projects and more tools per project was an exponential increase in the number of project-tool integrations. This necessitated a change in the way organizations adopted DevOps tools. This evolution took place in four phases.

Phase 1: Bring Your Own (BYO): Disparate tools

In the Bring Your Own DevOps phase, each team selected its own tools. This approach caused problems when teams attempted to work together because they were not familiar with the tools of other teams.

Phase 2: Best In Class (BIC): Standardized toolchain

To address this, organizations moved to the second phase, Best in Class DevOps. In this phase, organizations standardized on the same set of tools, with one preferred tool for each stage of the DevOps lifecycle. It helped teams collaborate with one another, but the problem then became moving software changes through the tools for each stage.

Phase 3: Do It Yourself (DIY): Custom integration

To remedy this problem, organizations adopted "Do It Yourself DevOps," building on top of and between their tools. They performed a lot of custom work to integrate their DevOps point solutions together; however, since these tools were developed independently without integration in mind, they never fit quite right. For many organizations, maintaining DIY DevOps was a significant effort and resulted in higher costs, with engineers maintaining tooling integration rather than working on their core product.

Phase 4: DevOps Platform: Single application

A platform approach was needed to improve the team experience and to improve business efficiency. GitLab, The DevOps Platform, replaces DIY DevOps, allowing visibility throughout and control over all stages of the DevOps lifecycle.

By empowering all teams - Software, Operations, IT, Security, and Business - to collaboratively plan, build, secure, and deploy software across an end-to-end unified system, GitLab represents a fundamental step change in realizing the full potential of DevOps. The DevOps Platform is single application powered by a cohesive user interface agnostic of self-managed or SaaS deployment, built on a single codebase with a unified data store allowed organizations to resolve the inefficiencies and vulnerabilities of an unreliable DIY toolchain.

As we look ahead to software-led organizations becoming even more distributed and agile, every company will need The DevOps Platform to modernize software development and delivery. By making it easier and trusted to adopt the next generation of cloud-native technologies - from microservices to serverless and eventually edge architecture - all companies will be empowered to ship software faster, at maximum efficiency, with security embedded across their end-to-end software supply chain.

Messaging Guidelines

| Category Name | The DevOps Platform |

|-----|-----|

| Category Definition | A platform delivered as a single application with one user interface, a unified data store, and security embedded within the DevOps lifecycle. |

| Category Rationale | As every company realized they needed to become a software company to remain competitive, the DevOps industry expanded, the number and complexity of tool-project integrations within an organization expanded. This was the result of two developments in DevOps:
1. Companies moved from monolithic architectures to microservices architectures. By doing so, applications could scale independently, allowing teams to move faster.
2. The faster delivery of software also required companies to use more DevOps tools per project.
The linear growth of both or more projects and more tools per project led to an

exponential increase in the number of project-tool integrations.

This resulted in longer cycle times and reduced productivity while increasing software vulnerability. To solve this, we need a platform to consolidate the tools that span the DevOps lifecycle. |

| Single Sentence (Biz/Tech Leaders) | GitLab is The DevOps Platform that empowers organizations to maximize the overall return on software development by delivering software faster, more efficiently, while strengthening security and compliance. |

| Single Paragraph (Biz/Tech Leaders) | GitLab is The DevOps Platform that empowers organizations to maximize the overall return on software development by delivering software faster and efficiently, while strengthening security and compliance. With GitLab, every team in your organization can collaboratively plan, build, secure, and deploy software to drive business outcomes faster with complete transparency, consistency and traceability. |

| Single Sentence (Engineers) | GitLab is The DevOps Platform, which combines the ability to develop, secure, and operate software in a single application that is easier to use and leads to a faster cycle time. |

| Single Paragraph (Engineers) | GitLab is The DevOps Platform, which combines the ability to develop, secure, and operate software in a single application that is easier to use and leads to faster cycle time. With GitLab, every team in your organization can collaborate in a single tool to manage the end-to-end development lifecycle, delivering more value faster, increasing visibility, and removing context switching. |

| Key Values | Value 1: Increase Operational Efficiencies | Value 2: Deliver Better Products Faster | Value 3: Reduce Security & Compliance Risk |

|-----|-----|-----|-----|

| Promise | Efficiency:
Maximize R&D and IT productivity. | Velocity:
Accelerate speed to market. | Security:
Compliance without compromise. |

| Customer pain | Organizations waste time and resources maintaining toolchain integrations, create silos of tool-specific competencies, create "Data gaps" in integrations resulting in incomplete context and manual workarounds. | DIY toolchains limit visibility into critical DevOps data that should inform decisions. They are also unreliable and inconsistent, prohibiting organizations from executing on decisions at scale without dramatically increasing risk. | Security scanning happens too late--if at all--requiring extensive rework as developers trace vulnerabilities back to their source and repair both the vulnerable code and other code changes built upon it. |

| GitLab Promise | As a single application, built on a unified data source, GitLab provides a complete DevOps solution without the need to spend any resources on building and maintaining a DIY toolchain. If integrations are necessary for any reason, businesses can integrate once and have access to the entire platform. | GitLab's single application, built on a unified data source, provides a complete end-to-end view of the entire software supply chain, enabling full transparency. The lack of toolchain integrations creates a more consistent, predictable, lower-maintenance development flow. | With built-in security scanners, organizations can scan every line of code as it is committed, allowing developers to identify and remediate vulnerabilities before they are even pushed. |

| ROI Theme | As a single application, built on a unified data source, GitLab provides a complete DevOps solution without the need to spend any resources on building and maintaining a DIY toolchain. If integrations are necessary for any reason, businesses can integrate once and have access to the entire platform. | GitLab's single application, built on a unified data source, provides a complete end-to-end view of the entire software supply chain, enabling full transparency. The lack of toolchain integrations creates a more consistent, predictable, lower-maintenance development flow. | With built-in security scanners, organizations can scan every line of code as it is committed, allowing developers to identify and remediate vulnerabilities

before they are even pushed. |

| Product Proof | - One system to install and maintain. - End-to-end lifecycle support from one provider. - One point of integration for external systems. - One user interface for the entire lifecycle. | - Value Stream Analytics allows you to monitor and manage the flow of work, identifying inefficiencies and blockers to value delivery. - Automatic correlation of artifacts and actions across the lifecycle--from ideation to code creation to testing, security scans, and performance impact--complete traceability. - Role-based dashboards (e.g., Security Dashboard, Epic Roadmaps) with contextual drill-downs into any level of detail | - Run static and dynamic security scans on every commit--without having to merge code. - Provide clear accountability for the introduction, remediation, and dismissal of security vulnerabilities and policy violations. - Maintain complete context by collaborating in a single system across development, operations, security, and audit/compliance. |

| Customer Examples | Glympse (Replaced 20 distinct tools with GitLab), Jasper Solutions (33-37% YoY Cost Savings, 30% reduction in cycle time, 25% increase in deployment frequency, 90-95% of projects on budget on time) | Goldman Sachs (From biweekly builds to > 1000/day) BI Worldwide (10X daily deployments for modernized applications, Eliminated challenges of multiple tools, logins, and user experiences) | HackerOne(Scanning earlier, more frequently, more economically) |

Supporting Data

In their Market Guide for DevOps Value Stream Delivery Platforms, Gartner's Strategic Planning Assumption was:

"By 2023, 40% of organizations will have switched from multiple point solutions to DevOps value stream delivery platforms to streamline application delivery, versus less than 10% in 2020."

_{Market Guide for DevOps Value Stream Delivery Platforms, Manjunath Bhat, Hassan Ennaciri, Chris Saunderson, Daniel Betts, Thomas Murphy, Joachim Herschmann, 28 September 2020}

_{Gartner does not endorse any vendor, product or service depicted in its research publications, and does not advise technology users to select only those vendors with the highest ratings or other designation. Gartner research publications consist of the opinions of Gartner's research organization and should not be construed as statements of fact. Gartner disclaims all warranties, expressed or implied, with respect to this research, including any warranties of merchantability or fitness for a particular purpose.}

GitLab believes this indicates that the market for a DevOps platform delivered as a single application will grow much faster than the DevOps market as a whole.

<iframe width="100%" height="500" src="https://www.youtube.com/embed/-CDU6NFw7U" frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-picture" allowfullscreen></iframe>

Segment messaging and positioning

Below are summaries of the value different segments will find in the DeOps Platform use case and suggestions for how to position GitLab to best explain those values.

SMB

All SMBs are dealing with some form of resource constraints as they grow. A DevOps platform will provide a growth path with the smallest amount of friction as they grow into the company they aspire to become.

Some SMBs will have a very clear idea of what they want from a DevOps solution, while others are just starting on their journey. For businesses in the former category, a DevOps platform provides a guide toward a productive solution that anticipates their needs as they grow, supported by evidence from successful companies of all sizes using the same solution, including some of the world's largest companies. A DevOps platform can guarantee minimal headaches from problems they have yet to encounter.

SMBs who are just starting with DevOps have likely already begun the process of integrating multiple tools, and are likely already resource constrained. These organizations are very aware of the time they waste on maintaining and integrating point solutions, and they would probably love to focus on more productive work. They will eventually need to know that a DevOps platform can both meet their criteria and stand up to point solutions--particularly in Source Code Management and Continuous Integration. As with the other category of SMBs, customer evidence from relevant larger companies will be important third-party proof points.

Mid-Market

Mid-Market companies are large enough and mature enough to have experienced the same challenges as larger enterprises, and much of the messaging mentioned in the Enterprise entry will generally resonate. Efficiency is essential, freeing up resources to develop more value, more quickly. Collaboration across the organization is also critical.

The key difference between the Mid-Market and Enterprise segments is the growth imperative. Mid-Market businesses are highly oriented toward growth and innovation, as they cannot yet compete on pure scale with larger competitors, but are large enough to be outmaneuvered by nimble startups. As such, they are very interested in moving quickly, but also wary of lock-in or poor scalability that could ultimately cap their upward or outward growth. To these businesses, proof points from larger businesses--particularly those of competitors or those from similar industries--are extremely helpful. The ability to adopt a platform on the fly--one component at a time, as opportunities arise--will be well-received.

Enterprise

Enterprises will already be familiar with the pains of the multi-product integration "toolchain tax." Those who have been pursuing a more aggressive DevOps strategy will likely be spending a large amount of their resources building and maintaining more than one toolchain, while others may have scaled back their ambitions because of complexity and cost.

In either case, the efficiency of a single, end-to-end platform will resonate across all roles. Allocating fewer resources to maintenance and integration means less waste and more focus on productive, interesting outcomes. Additionally, a DevOps platform allows enterprises to improve the quality of the work they do by increasing visibility and collaboration and encouraging a deeper engagement in all parties with the SDLC. Planners and business stakeholders can trace their requirements and suggestions all the way to their business impact after deployment, while developers and operations and security professionals can understand and contribute to the context around business discussions. Security, compliance, and audits have complete

traceability back to the source of irregularities.

Single-platform visibility and collaboration dovetails nicely with transformation initiatives, as well. As one platform for end-to-end DevSecOps, GitLab can become the engine of transformation from project- to product-based thinking, waterfall- to Agile-based methodologies, or other cultural and process transformations.

Elevator pitches per segment and persona

Below are one-sentence summaries of value you can deliver to user and buyer personas to position relevant aspects of the DevOps Platform use case's value, based on the market segment of the persona's company.

SMB

| User: Delaney - the Development Team Lead | User: Devon - the DevOps Engineer | Buyer: CTO— | Buyer: CIO— |

|---|---|---|---|

| Receive immediate feedback on the quality, performance, and security of code as soon as you commit. Collaborate seamlessly among teams. | Grow without having to build and support custom integrations. | Collaborate in a single system, minimize context-switching, and increase developer productivity and focus. | Grow without building and supporting custom integrations. Scale and manage a single system. |

— CTO (Dev, external focus) and CIO (Ops, internal focus) personas in development and may vary per segment.

</div>

Mid-Market

| User: Delaney - the Development Team Lead | User: Devon - the DevOps Engineer | Buyers: range from Erin - the Application Development Executive to the CTO— | Buyers: range from Kennedy - the Infrastructure Engineering Director to the CIO— |

|---|---|---|---|

| Receive immediate feedback on the quality, performance, and security of code as soon as you commit. Collaborate seamlessly among teams. | Increase reliability and eliminate ad hoc, team-based integrations. | Collaborate in a single system, minimize context-switching and waiting, identify and remove productivity blockers, and deliver more value faster with more productive, focused developers. | Increase reliability and performance while you grow by eliminating custom integrations. Scale and manage a single system. |

— CTO (Dev, external focus) and CIO (Ops, internal focus) personas in development and may vary per segment.

Enterprise

<div class="panel-body">

| User: Delaney - the Development Team Lead | User: Devon - the DevOps Engineer | Buyer:

Erin - the Application Development Executive | Buyer: Kennedy - the Infrastructure Engineering Director |

|---|---|---|---|

| Receive immediate feedback on the quality, performance, and security of code as soon as you commit. Collaborate seamlessly among teams. | Increase reliability and eliminate ad hoc, team-based integrations. | Collaborate in a single system, minimize context-switching and waiting, identify and remove productivity blockers, and deliver more value faster with more productive, focused developers. | Increase reliability and performance while you grow by eliminating custom integrations. Scale and manage a single system. |

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/devsecops/_index.md

Looking for a customer-facing overview of GitLab's DevSecOps capabilities? See the DevSecOps Solution

GitLab field teams refer to this Security and Governance highspot page for latest information

The page below is intended to align GitLab sales and marketing efforts with a single source of truth for our go-to-market efforts around DevSecOps.

Who to contact

	Product Marketing		Developer Advocate	
	-----		-----	
	Brian Mason (@BrianMason)		Fernando Diaz (@fjdiaz)	

The Market Viewpoint

DevSecOps

The DevSecOps usecase is applicable for customers who are trying to "shift left" to find security vulnerabilities earlier within their DevOps methodology but have not been able to achieve expected results.

Application Security is hard when security is separated from your DevOps flow. Security has traditionally been the final hurdle in the development life cycle. Iterative development workflows can make security a release bottleneck. Customers don't have enough security people to test all of their code, and hiring more security analysts won't automatically reduce the friction between app sec and engineering teams. Only testing major releases, or limiting tests to certain apps, leaves weak spots hackers can exploit. They need a way to balance risk and business agility. Instead of waiting for security at the end of the development process, they want to include it within their DevOps workflow. Often this is referred to as DevSecOps.

DevSecOps integrates automated security scans and compliance controls within the CI pipeline. GitLab takes this approach further by seamlessly embedding security and compliance within the DevOps platform, providing simplicity, visibility, and control.

Why is DevSecOps needed?

Security pros will always be outnumbered by developers. To scale application security efforts, developers must be enabled to find and fix software security vulnerabilities on their own, while they are working on their code. In addition, while the need for compliance and auditability has always been important, these requirements have greater attention following high-profile attacks on software supply chains and the related US President's Executive Order to improve cyber security.

Application security testing is still foundational, while now visibility and control across the entire software factory is even more paramount. Think of this as DevSecOps 2.0. GitLab's platform approach to DevOps provides a clear advantage to meet these new, broader DevSecOps requirements.

Desired business outcomes

- Shift left to find and fix vulnerabilities earlier when it's less costly to do so
- Already tried to shift left with incumbent tools - doesn't scale - looking for a simpler solution
- Reduce risk and technical debt by not introducing vulnerabilities into production
- More predictable cost of app sec in a DevOps, high velocity, iterative environment
- More secure software supply chain with better end-to-end governance to reduce security and compliance risks

Personas

User Personas

Users include both the developer and the security pro. We pride ourselves on having a united view of the software vulnerabilities and their status toward resolution.

The Developer uses GitLab primarily within the MR pipeline report

The developer cares about security but does not want to become a security expert. Their primary driver to write secure code is to protect their personal/professional reputation. They don't want to be the one that brings their company down via vulnerable code that they wrote. At the same time, they are goaled mostly on quickly turning out code that meets their users' requirements. Often they are not measured on security flaws. Security can seem like a necessary nuisance. Tools that fit within their workflow, without context-switching are most acceptable. The clarity GitLab provides by reporting vulnerabilities at code commit is helpful.

Amy the Application Security Engineer uses GitLab primarily in the Security Dashboard and vulnerability reports

The security pro cares most about managing risk to the enterprise/agency. They take a broad view of process looking for process improvement areas to reduce risk and avoid repeat mistakes. Because they care about risk, they want to identify unresolved vulnerabilities, their severity, and their remediation status. They care about trends over time and aggregate improvements. Often their metrics are mean time to remediation. It is rare that the security person themselves is able to remediate a software security flaw; they depend upon the developer to do

this. This goal misalignment is often a reason for contention between the groups. In traditional app sec environments, where testing is done at the end of the SDLC, they may spend a lot of their time tracking and reporting vulnerability statuses, vetting findings, and triaging to dev teams. Where development is more automated, they may be able to focus more on setting policies and allowing the tools to enforce them. They often want to avoid moving any new critical/high vulnerabilities into production and favor breaking the build to enforce this.

While developers and DevOps teams like to use GitLab for security, the security pro is often skeptical, comparing it to their favorite incumbent scanner. They may have built up a complex dashboard of their own and have a career invested in their favorite scanner. They are often reluctant to replace it, even if it can simplify their work as well as that of the developer.

Buyer Personas

The Security Manager or CISO (Sam's boss) is usually the buyer for the Ultimate tier

The security leader may have bet their career on justifying a very expensive scanner like Fortify or Veracode and are often reluctant to replace it, even if it can simplify work for dev and sec.

The key to winning their hearts is to focus on Simplicity and control

- Complexity is one of the CISO's chief complaints. Using one tool to provide a breadth of scan results in one place, without ongoing integration/maintenance, can greatly simplify their efforts.
- CISOs likely feel out of control, or at least pressured to be secure amidst evolving threats, high-profile cyber attacks, new compliance concerns, and development tool sprawl. It's hard to manage these software risks. Take control of your software development with a single platform that provides end-to-end simplicity, visibility, and control for all of your software development needs - choose GitLab.

Industry Analyst Coverage

2021 Gartner Magic Quadrant for Application Security Testing

Market Requirements (in priority order)

Market Requirements	Description	Typical capability-enabling features	Value/ROI
Common compliance controls	Controls necessary to protect the integrity of the software development and deployment process	Role-based access, MR approvals, and many others	Simplify audit and compliance and reduce risk of noncompliance.
Scan results for the Developer	For developers to find and fix vulnerabilities while they are coding, vulnerability scan results must be made available to the developer within their native workflow. Findings that can be automatically corrected should use automation to apply a fix and test the results.	Incremental scanning for rapid, iterative scans. Integrate scan results into the CI pipeline. Auto Remediation to automatically create a fix, eliminating developer effort.	Allows security flaws to be fixed early, when less expensive, removes context-switching, and minimizes risk by preventing vulnerabilities from reaching production.
Application Security Testing of code and components	Often referred to as Software		

Composition Analysis (SCA), it tests all code components (custom code and open source), wherever it resides (e.g. within containers). The GitLab survey shows Dependency scanning is most frequently used scan type. | SAST, Dependency Scanning, Container scanning, Secrets Detection, and (optionally) License Compliance | Reduces security and compliance risks. | Application Security Testing of running app | Scans that look for vulnerable code behavior as the code executes. | DAST, IAST, Input Fuzzing, UEBA/Threat modeling, Mobile app sec testing | Reduces security and compliance risks. |

| Security Governance | The solution automatically applies security policies against code to ensure that only appropriate risks are taken. Application vulnerabilities, representing risk, are tracked, managed, and reported. The solution enables routine assessments of security practices to evaluate for risk, compliance, audit and process improvement opportunities (usually for education purposes). | Security policy automation, Risk and compliance reporting, Audit reporting, Variety of security metrics and process reporting, Vulnerability database and management | Efficiently monitor, manage and mitigate risk. Ability to identify exceptions and refine policies over time. |

| Security guardrails (Preventative - Pre CI/CD) | Preventative Application Security uses guardrails to help teams consistently build things that are secure from the start. | Spell-check-like functionality that identifies insecure phrases as the developer types, bill of materials that limit developer choices to pre-approved code libraries, and auto-discovery that catalogs all third party code. | Prevents creating new vulnerabilities. |

| Works with existing and diverse tools | Security scanners must integrate into an existing environment that may use third party CI and/or security scanners. | APIs and Plug-ins | Allows continued use of existing investments and avoids a rip-n-replace scenario. |

The GitLab Solution

How GitLab Meets the Market Requirements

GitLab DevSecOps use case overview

```
<figure class="videocontainer">
<iframe width="560" height="315" src="https://www.youtube.com/embed/XnYstHObqIA"
frameborder="0" allow="accelerometer; autoplay; encrypted-media; gyroscope; picture-in-
picture" allowfullscreen></iframe>
</figure>
```

| Market Requirements | How GitLab Delivers | GitLab Category | Demos |

| ----- | ----- | ----- | ----- |

| Common compliance controls | GitLab provides many common controls throughout the SDLC | Access and Compliance within the Manage stage | Compliance pipelines |

| Scan results for the Developer | Incremental scanning for rapid, iterative scans. Integrate scan results into the CI pipeline. Auto Remediation to automatically create a fix. | CI (for MR pipeline), Auto remediation | Adding Security to your CICD Pipeline |

| Application Security Testing of code and components | SAST, Dependency Scanning, Container scanning, Secrets Detection, and License Compliance | SAST, Dependency Scanning, Container scanning, Secrets Detection, and License Compliance | SAST and Secret Detection
 Dependency scanning
 Container scanning
 License compliance
 Mobile SAST |

| Application Security Testing of running app | GitLab uses its review app, spun up during CI, to run dynamic application security tests. Recent acquisitions will provide multi-dimensional fuzz testing, useful for API scanning. Mobile app sec testing can be performed on any app within our language scanning capabilities | DAST, Input Fuzzing, Mobile app sec testing

(partners needed) | DAST |
Security Governance	Security Policy Automation, Compliance Assessment, Security Risk Assessment, Audit Assessment, Security Process Improvement/ Assessment, Vulnerability Management, Advisory Database	Security Dashboards, Vulnerability Management, Audit events Compliance Management	Accelerate AppSec Efficiency with the GitLab Security Dashboard
Security guardrails (Preventative - Pre CI/CD)	GitLab provides bill of materials showing dependencies. We do not yet use this to limit developers to use only pre-approved dependencies	bill of materials feature	Manage your Application Dependencies with GitLab
Works with existing and diverse tools	GitLab allows the integration of external CI tools to run alongside GitLab's CI enabling Jenkins users to also use GitLab Secure capabilities. Similarly, users may have invested in third party security scanners or may require another scanner for a language not covered by GitLab. We have made it easier for third party scanners to integrate into the GitLab MR and the security dashboards.	CI (for MR pipeline), Security Dashboards	Using GitLab Application Security Capabilities with Jenkins Creating Jira issues from GitLab vulns

Top Differentiators

Differentiator	Value	Proof Point	Demos
Detailed and Actionable Scan Results Displayed in MR created from Feature Branch	GitLab performs security scans like SAST, license compliance, dependency scanning before the code is merged - giving Developers opportunity to identify and fix security vulnerabilities before they context switch to other activities. This improves cycle time and development costs as the time and cost to resolve defects and vulnerabilities exponentially increase the later it is detected in the development cycle	Gartner - Integrating Security Into the DevSecOps Toolchain explains how Security should be included in the DevSecOps lifecycle in small actionable steps that developers can take action on quickly & integrating into defect tracking workflow to match the pace of security fixes to the pace of development.	Security Scans as Displayed in DevSecOps Overview
Block MR based on Security Policy	Bring Development and Security Teams closer by allowing security teams to apply organizational security policies before hand and review/approve security exceptions before the code is merged	-	Merge-Request Approvals as Displayed in DevSecOps Overview
Compliance Management	GitLab makes compliance easier by providing a single source of truth for Dev, Sec and Ops through a single data-store. Everything is audited and for every change, there is a single thread that contains the full audit log of every decision and action - making audit compliance a breeze	The auditor for Glympse observed that the company had remediated security issues faster than any other company that he had worked with before in his 20-year career. Within one sprint, just 2 weeks, Glympse was able to implement security jobs across all of their repositories using GitLab's CI templates	Manage Compliance with GitLab
Coverage-Guided Fuzz Testing	GitLab provides using contextual information from source code to better inform fuzz tests as well as to help correlate the results of a fuzz testing crash directly to the region of code that was vulnerable. This dramatically improves the cycle time to go from an initial fuzz test to a crash to an update to vulnerable areas.	-	Finding Bugs with Coverage Guided Fuzz Testing
Offline Environments	GitLab provides a variety of scanners to run in offline or limited connectivity environments. This feature enables security vulnerabilities to be detected in code that lies in offline environments.	-	Running GitLab Security Scans in Limited Connectivity and Offline Environments

How the GitLab DevOps platform helps achieve DevSecOps

Simplicity/efficiency

- Every piece of code is tested upon commit for security threats, without incremental cost, providing clear accountability for remediation.
- The developer can remediate now, while they are still working in that code, or create an issue with one click.
- Vulnerabilities are captured as a by-product of software development, and can be managed by security, with a single source of truth for better collaboration.
- A single platform avoids cost and effort to buy, integrate and maintain point solutions throughout the DevOps pipeline.

Visibility

- The dashboard for the security pro provides early insight to vulnerabilities not resolved upon merge.
- A single platform improves allows the organization to see who changed what, where, and when across the entire SDLC.

Control

- Security policies can be automatically applied to all pipelines for consistency, compliance, and simplified audits
- Role-based access controls provide separation of duties and protect against malicious insider threats and accidental events
- A breadth of additional compliance controls provide superior governance over software development, delivery, and use.

What Are The GitLab Advantages?

Platform approach. With a single platform for the entire SDLC, GitLab enables simplicity of embedded security scanning along with end-to-end visibility and control not possible with point solutions. Vulnerabilities can become issues for follow up with one click. Remediation status is always evident. Changes to the code and to the cloud native environment upon which it depends is tracked. When using GitLab, no additional integration is needed between app sec and ticketing, CI/CD, etc.

Congruent with DevOps processes. Unlike traditional application security tools primarily intended for use by security pros, GitLab's security scanning is built into the CI/CD workflows where the developers live. We empower developers to identify vulnerabilities and remove them early in the development cycles. At the same time, we provide security professionals the ability to evaluate, triage, and manage vulnerabilities not resolved by the developer, across projects and groups.

Comprehensive for modern applications. GitLab Ultimate includes not only SAST, DAST, and dependency scanning (which are table stakes for any software security program), but also comprehensive scanning critical for today's cloud native applications. Scanning includes Infrastructure-as-code (IaC), APIs, containers, Kubernetes cluster images, and fuzz testing. All of these represent new attack surfaces that must be secured as part of your software supply

chain.

Consistency and control. In addition to automated scans, it's important to automate policies to provide guardrails that allow developers to run fast but run safely. Common controls for compliance, along with approval rules, and compliant pipeline workflows that are centrally administered all work together to help you protect your software supply chain.

Message House

The message house for DevSecOps provides a structure to describe and discuss the value and differentiators for the use case.

Competitive Comparison

See how we compare against other Security tools

How our governance compares:

1. Role-based access control (RBAC) for separation of duties. Competitive products's roles are broader and when a person changes roles, his/her permissions must be changed manually. Why is this important? If someone has access to push to prod and is demoted or moves to another group, you'd want the permissions to change automatically to avoid insider threats.

1. Our workflows include compliance within MR approvals. No manual checks that impact velocity. (In essence, we shift left compliance also.)

1. External status checks is an important feature for regulated industries. Changes are approved and must be pushed to production within a given timeframe. Delays can cause the approval process to start over.

1. With GitLab we have projects and groups where projects inherit policies from the group. Competitors cannot structure policies as flexibly as GitLab, an important feature for enterprise users. Examples include group level runners.

1. Compliant pipelines allow GitLab users to select their compliance framework (e.g. PCI, HIPPA, etc) and those policies are used - and the developer cannot disable it (due to RBAC)

Key Value by tier

Free and Premium

Why choose GitLab free or Premium for DevSecOps? Security matters to everyone, and we're committed to lowering the barriers to a fully secure, compliant SDLC. That's why we've migrated Brakeman SAST scanning and secrets detection to Free, developers at every product tier to scan their source code for known vulnerabilities.

Key DevSecOps features with Free/Premium:

- Static Application Security Testing - check for potential security issues by evaluating static code.
- Secrets Detection - avoid exposing secrets and credentials for potential exploit.

Ultimate

Why choose GitLab Ultimate for DevSecOps? Achieve advanced DevOps maturity with enterprise-level application security capabilities.

- Vulnerability management via vulnerability reports and Security dashboards at the project, group, and instance level provide visibility into application risks earlier in the SDLC, as well as visibility into their remediation status.
- Security Approvals in Merge Requests
- GitLab Secure helps you shift left to enable developers to find and fix vulnerabilities early. Multi-faceted scanning includes SAST, DAST, Container and Dependency scanning, Secrets Detection, coverage-guided fuzz testing, API fuzzing, and License Compliance checks, all within the GitLab Merge Request CI Pipeline with results provided to the developer at the point of code commit. Vulnerability findings are actionable and provide remediation advice while the developer is still working on the code.
- More enterprise-level compliance controls

In addition, enjoy these benefits of Ultimate:

- Enterprise-grade priority support, including 24/7 uptime support, a named Customer Success Manager (CSM), and upgrade assistance are all still included with Ultimate.
- Embed security and compliance into your CI Pipelines.
- Protect your IP and get access to free guest users.

Key DevSecOps features with Ultimate:

- Dynamic Application Security Testing - analyze review applications to identify potential security issues.
- Dependency Scanning - evaluate third-party dependencies to identify potential security issues.
- Container Scanning - analyze Docker images and check for potential security issues.
- Security Dashboard - visualize security status for projects.
- Vulnerability Management
- License Compliance - identify the presence of new software licenses included in your project and track project dependencies. Approve or deny the inclusion of a specific license.
- Compliance Dashboard - See if merge requests were approved, and by whom.

Technology Partnerships

We partner with key industry vendors to extend GitLab's ability to address customer needs and fulfil the market requirements.

One of the first partners to integrate their scan results into the GitLab Security Dashboard and the GitLab CI pipeline is WhiteSource.

- They extend scanning language support to nearly 200 additional languages and provide deeper dependency insight.
- Learn more about how to use WhiteSource within GitLab and more;
- Watch [Dependency Scanning with GitLab and WhiteSource](#) to get started.

A more complete list of technology partners can be found on our [security partners page](#).

If you or your customer has a third party they'd like to see integrated into GitLab, send them to the partner integration page for instructions.

Selling the DevSecOps Solution

Customer Facing Slides

- Security customer presentation

Discovery Questions

The suggested discovery questions below are meant to help you uncover opportunities when speaking with prospects or customers who are not currently using GitLab for Secure/Protect. They are grouped by topic or entry point. Don't try to use them all, just those most relevant to your customer. The deeper you can dig into their processes, the more benefits you are likely to show of using GitLab. Feel free to contribute!

Initial probe for direction. Where's the pain?

Integrating application security testing into Agile DevOps software development is difficult with many potential challenges. Use these 6 questions to probe a bit to see which areas are of most concern, then go deeper on those topics further below.

1. Finding security vulnerabilities - Can your security scans keep up with your iterative coding velocity? How often are projects stopped waiting for a security scan?
2. Collaboration /visibility - Is there friction between dev and sec? Can you see security risks at any point in the SDLC?
3. Remediation - How much time is wasted translating what was found by sec to what needs to be done by dev and tracking if it was done? Can you quickly see the status of security remediations?
4. Managing Risk in a DevOps environment - What percentage of code are you currently scanning? Are there holes where an attacker could more easily enter and then traverse laterally? How much more would it cost you to scan all of your code?
5. Policy Automation - Are Security policies automated with security requirements built into the development process? Or are most projects secured a bit differently every time?
6. Tools - Is your organization investing to improve application security in the short term or long term? Is there a clearly defined strategy or timeline? Are you working to integrate app sec tools into the DevOps tool chain?

When speaking with C-levels, ask them about Governance and control challenges.

1. Finding security vulnerabilities

- How often do you rely on pen testing as your primary means of finding vulnerabilities? (indicates little automation; our out-of-the-box scanning can get them started quickly)
- Have you automated app sec testing within your DevOps process? Can you tell me more about this?
- What application security scans are most important to you? (SAST, DAST, IAST, dependency scanning, container scanning, secrets detection, license compliance? Mobile app testing? Fuzz testing?) Would you like to be able to run multiple types of scans for defense in depth? If you

are not today, what prevents it?

- What steps have you taken to enable developers to find and fix vulnerabilities themselves?

Are the scan results in the CI pipeline?

- How valuable would it be if your developers could find and fix more vulnerabilities themselves before the code ever leaves their hands?

- If you are using a security 'spell-checker' to aid developers, how effective is it? Are you still finding vulns with static testing after the code is merged? (the point is this alone is not enough)

2. Collaboration /visibility

- Are projects delayed due to inefficient handoff between teams, lack of visibility across systems, or lack of planning and consideration?

- Is time lost translating or reconciling between security tool findings, planning/development tools and issue tracking?

- Is there tension between responsibility for security and ability to improve it?

- How might your process be simplified if there were a single source of truth for both security and development to find and manage vulnerabilities?

- Can vulnerabilities be easily tracked by both developers and security to see where they reside in the code, who created them, action taken (issue created, dismissed, unresolved), and their remediation status? How time consuming is it to get this information?

3. Efficiency of remediation

- How many steps are required before vulns get to a developer who can remediate? How much time is wasted on these steps between development and security?

- What percentage of vulnerabilities do you think could be remediated by the developer right away if found while they were still working on the code? If it were more, would it save you time and money?

4. Managing Risk

- Are you mostly concerned about the OWASP Top 10. What if you could find the OWASP Top 10 for every code change and free your development workflow by decluttering their tasks to focus on vulns they just created and not technical debt from past efforts to avoid creating new technical debt? Would that help you avoid new technical debt and reduce risk?

- Do you focus on remediation of critical and high vulnerabilities? How much of your remediation is for medium and low vulns? Did you know that most exploits are against medium risk vulnerabilities? What if you could help developers find and fix each vuln as they are created?

- What percentage of code are you currently scanning? Are there holes where an attacker could more easily enter and then traverse laterally? How much more would it cost you to scan all of your code?

- If you are using containers, orchestrators, and/or microservices/API's, how are you scanning them for vulnerabilities and monitoring them during production?

- What percentage of vulns found that require remediation are actually remediated? How quickly? (Mean time to remediation)

- Can you see how developers handled vulnerabilities that were found? Would it be valuable to have visibility into vulnerabilities and their risk earlier in the lifecycle? Would this help you with security audits?

- How much of the security team's time is spent tracking vulnerabilities, triaging them, and following up to see that they were remediated?

5. Policy Automation

- Does automation enforce security standards? Are Security policies automated with security requirements built into the development process? Or are most projects secured a bit differently every time? How much manual intervention is needed?
- How difficult is it to evaluate compliance with policies and drift/exceptions?
- Compliance should be regularly evaluated and exceptions reviewed. Automating overly rigorous policies may prove detrimental to business objectives and may not be realistically achieved, so it is important to find a balance between compliance and efficiency.
 - Do you have Visibility into access control, reporting, and change logs?
 - Do you know how often policy exceptions are granted?

6. Tools

- How many different security tools are you using (which ones?)
- Have you integrated your security tools into your CI pipeline so that security scans run automatically or are they kicked off manually? Are the integrations fragile? Complex? Time consuming? How often does that process need to be updated? How much effort is required to set up for a new project? How do you ensure the scans are applied consistently? How often is your planned work interrupted to fix or maintain your security integrations?
- Which of your security scans are able to provide vulnerability findings directly to the developer? Can you scan a feature branch? (before code is merged) Are you able to see the line of code and which developer created the vulnerability? Do vulnerabilities automatically initiate a work ticket/issue creation?
- Have your teams run into roadblocks or hurdles automating security tests within the CI pipelines at scale?
- How predictable is the cost of your app sec tools? If you find more vulnerabilities, or test more apps, does it cost you more?

Potential Objections

I have an incumbent tool. How does your scanning compare?

GitLab scanners use a combination of proven open source scanners and proprietary scanners.

Finding vulnerabilities is important, but what you do with the results is equally important. Consider:

- Are your developers able to see vulnerabilities he/she created with every code commit or at least before merging their code with others?
- Are the scan findings actionable for the developer, within their native workflow?
- Is plugin management and compatibility an ongoing concern?

Also, how predictable is the cost of these tools? If you find more vulnerabilities, or test more apps, does it cost you more? Are you essentially penalized for more testing? How does that work with DevOps breaking apps into microservices and running more frequent pipelines? In fact, Gartner even called us out in a report on how to set up an AppSec program inexpensively (7 Tips

to Set Up an Application Security Program Without Breaking the Bank).

If using Fortify, Veracode or Synopsys

- Do your scan times keep up with or inhibit software iteration?
- Are you able to scan every code change?
- Are you overwhelmed by the number of findings? Do they all get remediated? If not, how do you prioritize and how much time/effort is required? What if the developer could prioritize, using the same guidelines as security, and fix vulns as he/she created them?
- Are you able to afford scanning all of your code?

If using Snyk, WhiteSource or other point solutions

- How are you scanning API's and containers? Are you able to monitor them in production?
- Are you scanning all of your code? At every commit? Does the developer have all of the information they need for remediation?

If using GitHub Actions and/or Azure DevOps

- How much effort is required to set up your security scans for every project?
- How difficult is it to enforce policies/standards for security scanning in every project/pipeline?
- How difficult is it to manage role-based permissions?
- Are governance controls as granular as you need them to be?
- Do you have visibility across all of the tools used in the marketplace to see who changed what, where, and when?

I can't justify the cost difference for Ultimate.

NIST demonstrated the cost savings from shifting security left back in 2002. How far left are you currently? Hypothetically, if 50% of your vulnerabilities could have been found by the developer and if half those could hypothetically be fixed before the code ever leaves the developer's hands, what value would that have for your cost and your risk exposure? Let's consider the potential benefit of:

- fewer vulns to prioritize, vet, triage, and track
- fewer vulns with their associated risk exposure
- less need to stop business innovation to fix security flaws after-the-fact

How predictable is the cost of other app sec tools? If you find more vulnerabilities, or test more apps, does it cost you more? Are you essentially penalized for more testing? How does that work with DevOps breaking apps into microservices and running more frequent pipelines? What is the value of scanning every code change going forward? What is the impact on your technical debt over time? What would it cost you with your other tools to scan every software change?

Proof Points - customers

Quotes and reviews

GitLab customer, HERE, shares their experience with using GitLab to Shift Left and also spoke

at GitLab Commit 2021.

GitLab customer, Arctic Engine, shares their experience with using GitLab's fuzz testing to find unknown vulnerabilities.

Gartner Peer Insights

Gartner Peer Insights reviews constitute the subjective opinions of individual end users based on their own experiences, and do not represent the views of Gartner or its affiliates. Obvious typos have been amended.

"GitLab Application Security Testing helps to analyze our source code for vulnerabilities, listed in GitLab Security Dashboard. It is easily configurable with Docker setup. It shows the complete vulnerabilities report immediately after a new merge request is created within a project. With listing potential risks in code, it also prioritizes vulnerabilities in terms of critical, high, low, medium which helps team to plan and focus on what first."

>

> - Sr. Software Engineer, Gartner Peer Insights Review

"The Application testing feature of [GitLab] is very useful in scanning for vulnerabilities in the applications. There are multiple test[s] available. We mostly use the tools to scan the docker containers, dependencies, source code and web application for the vulnerabilities."

>

> - Native Android Application Developer, Gartner Peer Insights Review

G2

- GitLab is a leader on the G2 Grid® for Static Application Security Testing (SAST)
- GitLab is a leader on the G2 Grid® for Dynamic Application Security Testing (DAST)

Customer Case Studies

HackerOne

- Problem: Velocity with security
- Solution: GitLab Ultimate
- Results: 5x deployment speed with GitLab security

Glympse

- Problem A complex developer tech stack with over 20 distinct tools that was hard to maintain and manage. Teams spent several hours a week keeping tools running rather than shipping innovation to their app.
- Solution: GitLab Ultimate (SCM, CI, DevSecOps)
- Results: ~20 tools consolidated into GitLab and remediated security issues faster than any other company in their Security Auditor's experience.
- Sales Segment: SMB

BI Worldwide

- Problem: Had a large legacy codebase that was created years ago. Looking to eliminate (or at least significantly reduce) manual testing and manual deployments to their on-premise infrastructure.
- Solution: GitLab Ultimate (SCM, CI, DevSecOps)
- Results: 10x daily deployments Simplified their toolchain and migrated from their previous Git tools to GitLab. Saw an immediate improvement of collaboration and release pace as a result of their move to GitLab.
- Sales Segment: Mid-Market

Chorus

- Problem: The founders of Chorus built the tool from the beginning using GitLab.
- Solution: GitLab Ultimate (SCM, CI, DevSecOps)
- Results: 6 week production cycles reduced to 1 week During a recent audit for SOC2 compliance, the auditors said that Chorus had the fastest auditing process they have seen and most of that is due to the capabilities of GitLab.
- Sales Segment: SMB

References to help you close

SFDC report of referenceable secure customers Note: Sales team members should have access to this report. If you do not have access, reach out to the customer reference team for assistance.

Adoption Guide

The following section provides resources to help CSMs lead capabilities adoption, but can also be used for prospects or customers interested in adopting GitLab stages and categories.

Playbook Steps

- Start with the first 6 Discover Questions to identify which area to drill into further.

Adoption Recommendation

There are multiple pathways to adopting GitLab's DevSecOps workflow, depending on the current state of the customers' current state. The following diagram shows the adoption sequence and relationship between scenarios.

This table shows the recommended use cases to adopt, links to product documentation, the respective subscription tier for the use case.

Feature / Scenario	Free	Premium	Ultimate
Product Analytics Notes			
-----	-----	-----	-----
-----	-----	-----	-----
Adopt GitLab Flow	X	X	X

Try / Utilize Auto DevOps					Partial	Partial	X	
Automated Testing with CI					X	X	X	
Only SAST at all tiers								
Review app					X	X	X	
Needed to run DAST in CI/CD pipeline								
Merge Request Approval Flow / Rules							X	X
counts.mergedmergerequestsusingapprovalrules								
Protected Environments						X	X	
Container Registry	X	X	X	containerregistryenabled				
Package Registry	X	X	X	countsmohtly.packages				
SAST (Static Application Security Testing)							X	X
X usersastjobs								
Secret Detection					X	X	X	
usersecretDetectionjobs								
DAST (Dynamic Application Security Testing)								
X userdastjobs								
Container Scanning							X	
usercontainerscanningjobs								
Dependency Scanning							X	
userdependencyscanningjobs								
License Compliance								
X userlicenseManagementjobs								
API Fuzzing								
X userapifuzzingjobs, userapifuzzingdndjobs on self-managed								
Coverage Fuzzing							X	
usercoveragefuzzingjobs								
Security Approvals					X			

The table includes free/community and paid tiers associated with GitLab's self-managed and cloud offering.

- F/C = Free
- S/P = Premium
- G/U = Ultimate

Additional Documentation Links

- From SCM and CI to Security (GitLab internal only)
- GitLab Security Compliance Controls
- GitLab Security Practices
- Security Planning

Enablement and Training

The following will link to enablement and training videos and content.

- Handling Security Audits
- Application Security at High Velocity
- Adding Security to your pipelines

Professional Service Offers

GitLab offers a variety of pre-packaged and custom services for our customers and partners. The following are service offers specific to this solution. For additional services, see the full service catalog.

- DevOps Fundamentals Training (all stages of the DevOps lifecycle)
- GitLab CI/CD Training
- Integration Services

Resources

White paper

- A Seismic Shift Left

eBook: Ten Steps Every CISO Should Take to Secure Next-gen Applications

- Gated
- Ungated

Integration with third party commercial scanners

- WhiteSource

Blogs

- How application security engineers can use GitLab to secure their projects
- How to capitalize on GitLab Security tools with external CI

Other DevSecOps Videos

- Deep Dive into a Security demo
- See how integration is the key to successful DevSecOps

Clickthrough & Live Demos

- All Marketing Click Through Demos
- All Marketing Live Demos

Roadmap

- Secure and Govern Roadmap

Technical Resources for Solution Architects

Sometimes customers and prospects have unique requirements, often around using an preferred scanning tool to integrating with some other part of their tool chain. If you or your customer has a third party they'd like to see integrated into GitLab, send them to the partner integration page for instructions. While GitLab can be a single platform to meet all of their needs, often they need an on-ramp to help them transition or proof of the integration before purchasing GitLab. The resources below may help.

NOTE: Please do not use these to put GitLab in the position where users expect us to support a 3rdparty product integration that we do not officially recognize.

- Using OSS Review as your default License scanner
- Integrating GitLab with Sonarqube

Buyer's Journey

Inventory of key assets in the buyer's Journey

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/devsecops/message-house.md

| Positioning Statement: | Simplify and scale security and compliance while adding visibility and control |

|-----|-----|
----|

| Short Description | GitLab automates robust application security scanning, embedded within CI, to help you align your security program to the DevOps development process. Automated security and compliance policies help developers go fast while managing software risk. |

| Long Description | The complexity of integrating security is one of the biggest challenges facing DevOps. GitLab simplifies DevSecOps efforts and improves compliance, by embedding robust security capabilities for both the developer and the security pro into one end-to-end DevOps platform. The DevOps platform approach improves visibility and control across the entire software factory. |

| Key-Values | Simplify governance of your software factory for security and compliance | Scan every code change before it leaves the developer's hands, simplifying and even automating remediation | Embed security, don't just integrate it |

|-----|-----|-----|
-----|

| Promise | End-to-end visibility and control with guardrails for security policies and common controls for compliance. | Use automation to find and fix security flaws within the developer's native workflow, providing clear accountability so developers know exactly what risk they introduced. | A single application for both the developer and security can avoid costly maintenance while providing a single source of truth for better collaboration. |

| Pain points | Protect the integrity of your software | Eliminate costly triaging and tracking of vulnerabilities that can be fixed at their source. | Stop managing complex tool chain plug-ins and fragile automation scripts. |

| Why GitLab | The platform approach enables comprehensive insight for who changed what, where, when for software and infrastructure-as-code | Robust security scans embedded within GitLab CI, including SAST, DAST, Dependency, fuzz testing, API fuzzing, and Container scanning, along with

License Compliance and Secrets Detection | The Security Dashboard and Vulnerability Report provides insights security pros need, showing remaining vulnerabilities across projects and/or groups, along with actions taken, by whom and when. |

| Proof points | Chorus uses GitLab for SAST... | Glympse auditors love the transparency GitLab provides.

SECTION: marketing/brand-and-product-marketing/product-and-solution-marketing/usecase-gtm/gitops/_index.md

Who to contact

Product Marketing	Developer Advocate
Daniel Hom (@danielhom)	@csaavedra1

The Market Viewpoint

The need for GitOps

Modern applications are developed with rapid iteration and run at highly dynamic scale. In an organization with a mature DevOps culture code can be deployed to production hundreds of times per day. Applications can then run under highly dynamic loads from a few users to milli